

Comment Toxicity Detection using NLP, Machine Learning, and Transformers

Project Overview

This project focuses on detecting toxic comments from user-generated text using Natural Language Processing (NLP), Machine Learning models, and Transformer-based Deep Learning models.

The main objective of this project is to classify online comments into multiple toxicity categories such as:

- Toxic
- Severe Toxic
- Obscene
- Threat
- Insult
- Identity Hate

This project implements both classical machine learning approaches and advanced transformer-based models to compare their performance for multi-label text classification.

Problem Statement

Online platforms and social media applications receive millions of comments daily. Many of these comments may contain:

- abusive language
- hate speech
- threats
- offensive content

Manual moderation becomes difficult at scale.

The goal of this project is to build an intelligent NLP-based system that can automatically detect toxic comments and help content moderation systems identify harmful text efficiently.

Project Objectives

The main objectives of this project are:

- Perform Exploratory Data Analysis (EDA) on toxic comment dataset

- Build custom text preprocessing pipeline
 - Implement TF-IDF feature extraction
 - Train and evaluate multiple machine learning models
 - Compare model performances
 - Implement Transformer-based BERT model
 - Build an interactive Streamlit application for real-time toxicity prediction
 - Allow users to enter comments and receive toxicity predictions instantly
-

Dataset Information

Dataset used:

Toxic Comment Classification Dataset

The dataset contains user comments along with multiple toxicity labels.

Dataset Columns

Column Name	Description
comment_text	User comment text
toxic	Toxic comment
severe_toxic	Highly toxic comment
obscene	Obscene content
threat	Threatening content
insult	Insulting language
identity_hate	Hate speech targeting identity

This is a Multi-Label Classification problem because a single comment can belong to multiple toxicity categories at the same time.

Technologies Used

Programming Language

- Python

Libraries and Frameworks

Data Manipulation

- Pandas
- NumPy

Data Visualization

- Matplotlib
- Seaborn
- WordCloud

NLP and Text Preprocessing

- NLTK
- Regular Expressions (re)

Machine Learning

- Scikit-learn

Deep Learning / Transformers

- Hugging Face Transformers
- PyTorch

Deployment

- Streamlit
-

Exploratory Data Analysis (EDA)

The following EDA steps were performed:

1. Dataset Inspection

- Checked dataset shape
- Checked dataset columns
- Displayed sample records
- Verified data types

2. Missing Values Check

- Verified that the dataset contains no null values

3. Duplicate Values Check

- Verified that the dataset contains no duplicate records

4. Toxic vs Non-Toxic Distribution

A new binary label column was created for EDA purposes:

- Toxic = 1
- Non-Toxic = 0

The dataset was found to be highly imbalanced.

Observation

- Majority of comments are non-toxic
- Toxic comments are significantly fewer
- This imbalance affects model learning and evaluation

5. Comment Length Analysis

Comment length feature was created to analyze the relationship between comment size and toxicity.

Observation

- Toxic comments generally vary in length
- Some extremely long comments exist as outliers
- Most comments are relatively short

6. Most Common Words Analysis

Most frequent words from:

- all comments
- toxic comments

were analyzed.

Observation

- Common stopwords appear frequently
- Toxic comments also contain conversational words
- Toxicity often depends on context rather than isolated words

7. Word Cloud Visualization

Word clouds were generated to visualize frequent words.

Observation

- Larger words represent higher frequency
 - Toxic datasets contain repeated aggressive language patterns
-

Text Preprocessing

A custom text preprocessing transformer was implemented using Scikit-learn compatible architecture.

Preprocessing Steps

The following preprocessing operations were performed:

- Convert text to lowercase
 - Remove URLs
 - Remove hashtags
 - Remove mentions
 - Remove email IDs
 - Remove special characters
 - Remove extra whitespaces
 - Tokenization
 - Stopword removal
 - Lemmatization
-

Custom Transformer

A custom transformer class was created using:

- BaseEstimator
- TransformerMixin

This transformer was integrated directly into the Scikit-learn pipeline.

Advantages

- Fully automated preprocessing
 - Reusable preprocessing pipeline
 - Cleaner workflow
 - Easy deployment integration
-

Feature Extraction

TF-IDF Vectorization was used for converting text into numerical features.

Why TF-IDF?

TF-IDF helps:

- identify important words
 - reduce importance of common words
 - improve text representation for machine learning models
-

Machine Learning Models Implemented

1. Multinomial Naive Bayes

Pipeline:

Text Preprocessing → TF-IDF → MultinomialNB

Observation

- Fast training speed
 - Poor performance on minority classes
 - Low recall and F1-score for rare toxicity labels
-

2. Logistic Regression

Pipeline:

Text Preprocessing → TF-IDF → Logistic Regression

Observation

- Significant improvement over Naive Bayes
 - Better balance between precision and recall
 - Suitable for probability-based predictions
 - Useful for Streamlit deployment
-

3. Linear Support Vector Classifier (LinearSVC)

Pipeline:

Text Preprocessing → TF-IDF → LinearSVC

Observation

- Best performing classical machine learning model
 - Higher F1-scores across multiple toxicity categories
 - Better handling of high-dimensional sparse TF-IDF features
 - Selected as the best ML model for comparison
-

Transformer Model - BERT

A transformer-based BERT model was implemented using Hugging Face Transformers.

Why BERT?

Unlike classical ML models, BERT:

- understands context
- captures semantic meaning
- handles complex language patterns
- improves contextual toxicity understanding

BERT Workflow

Raw Text → BERT Tokenizer → BERT Model → Multi-Label Predictions

Features

- Pretrained Transformer Model
 - Multi-label classification
 - GPU-based training
 - Context-aware predictions
-

Model Evaluation Metrics

The following evaluation metrics were used:

- Precision
- Recall
- F1-Score
- Support

Metric Definitions

Precision

Measures how many predicted toxic comments were actually toxic.

Recall

Measures how many actual toxic comments were correctly identified.

F1-Score

Harmonic mean of precision and recall.

Support

Number of actual occurrences of each class.

Best Model Selection

Model	Performance
MultinomialNB	Basic Performance
Logistic Regression	Good Performance
LinearSVC	Best Classical ML Performance
BERT	Advanced Transformer Performance

LinearSVC achieved the best classical machine learning performance.

Logistic Regression was selected for probability-based deployment.

BERT was implemented for advanced contextual understanding.

Streamlit Application

An interactive Streamlit dashboard will be developed for deployment.

Planned Features

- User text input
 - Real-time toxicity prediction
 - Probability score display
 - Toxicity category prediction
 - CSV upload support for bulk predictions
 - Model performance visualization
-

Challenges Faced

- Handling multi-label classification
 - Dataset imbalance
 - Minority class prediction difficulty
 - Computational complexity of BERT
 - Long training time for transformers
-

Future Improvements

Possible future enhancements:

- Hyperparameter tuning
 - Better class balancing techniques
 - Advanced transformer fine-tuning
 - Real-time API deployment
 - Larger toxicity datasets
 - Explainable AI visualization
-

Conclusion

This project successfully demonstrates the implementation of:

- NLP preprocessing
- Multi-label toxicity classification
- Classical machine learning models

- Transformer-based deep learning models
- End-to-end toxicity prediction workflow

The project highlights how NLP and Machine Learning can be used effectively for automated content moderation systems.

References

- Scikit-learn Documentation
 - Hugging Face Transformers Documentation
 - NLTK Documentation
 - Streamlit Documentation
 - Toxic Comment Classification Dataset
-

Author

Sangeetha S

NLP | Machine Learning | Deep Learning Project